Distributed and Enterprise Software Development (DESD)
UFCFTR-30-3

Week 8

# Distributed Systems

The foundation of everything we build this year

# Today

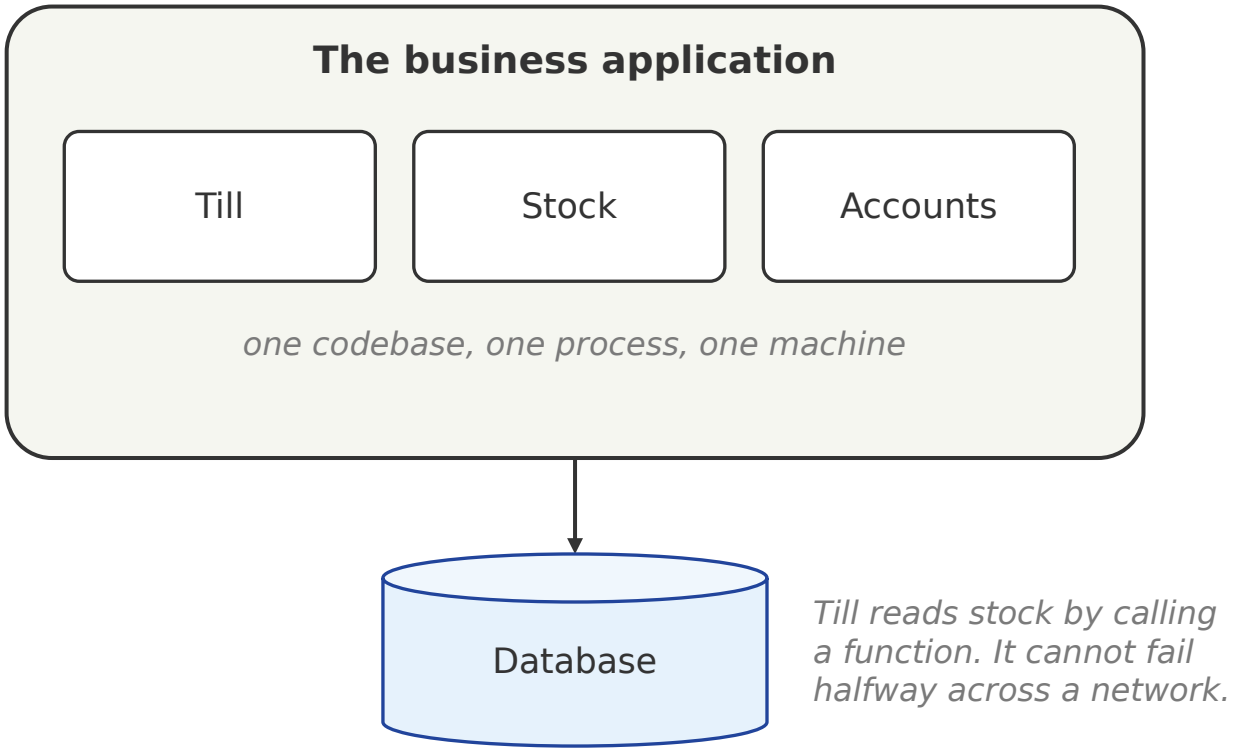
1. **Why systems become distributed**
   The business forces, not the technology
2. **How the parts talk**
   Synchronous, asynchronous, and who coordinates
3. **Architectural styles**
   Monolith, tiers, services, and how each one scales
4. **Data across machines**
   Replication, consistency, and transactions that span services
5. **What breaks, and how it becomes visible**
   Partial failure, observability and security

**Everything today points at one question: what does this cost?**

# Why systems become distributed

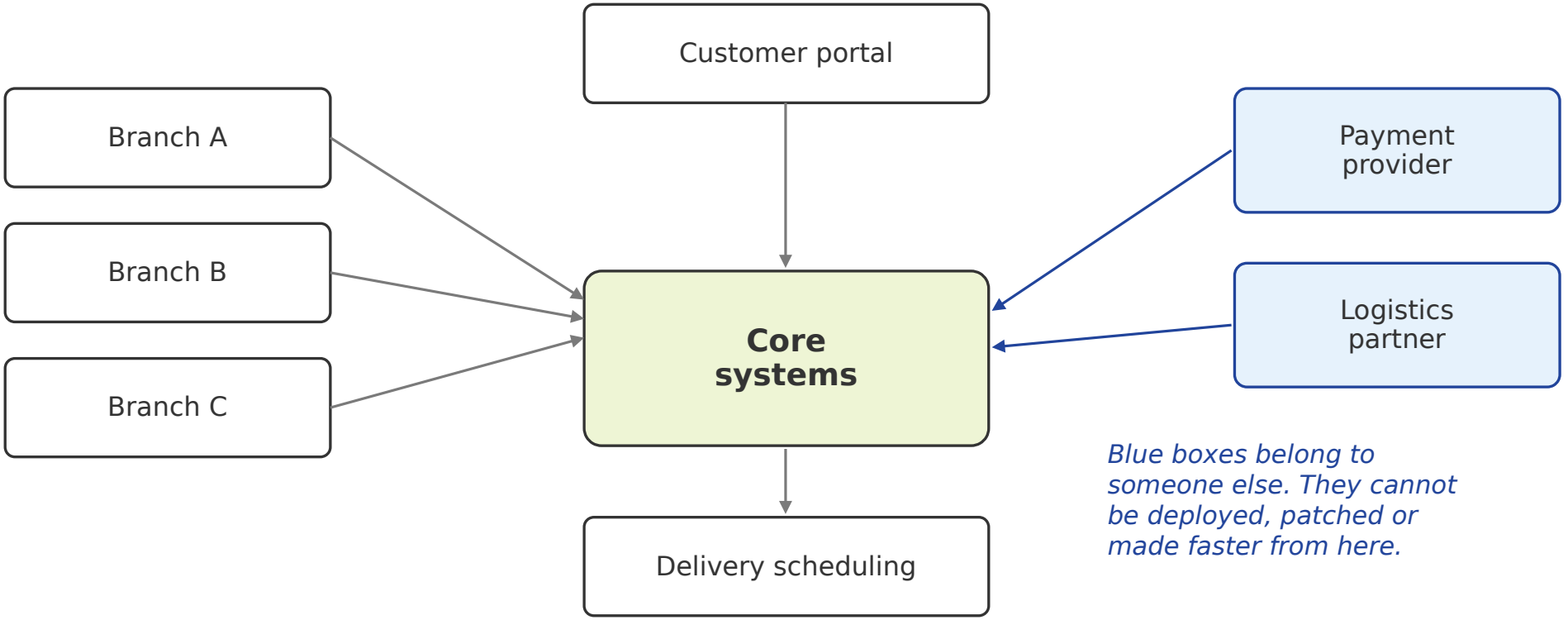
Nobody sets out to build a distributed system

# One business, one system




**This is not a primitive design. For one shop it is the correct one.**

# Then the business grows




**Every arrow crossing a machine boundary is a new way for the system to fail.**

# Four forces that distribute a system

### Scale

More work than one machine can carry, or more than one machine should risk carrying.

### Geography

Users, sites and data in different places, with a network in between.

### Ownership

Capabilities that belong to another team, another company, another contract.

### Failure isolation

One part must be able to fall over without taking the rest with it.

**Name the force before proposing the architecture.**

# So what is a distributed system?

"A collection of independent computers that appears to its users as a single coherent system."

*Tanenbaum and van Steen, Distributed Systems*

**Which commits the design to three things:**

**No shared memory**

parts exchange messages, nothing else

**No shared clock**

"before" and "after" stop being obvious

**Independent failure**

one part can die while the rest runs

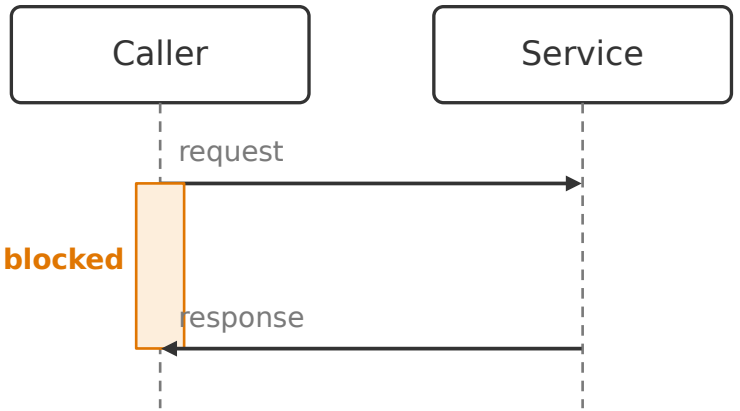
**The hard part is not the distribution. It is the "single coherent system".**

# How the parts talk

One distinction that recurs all year

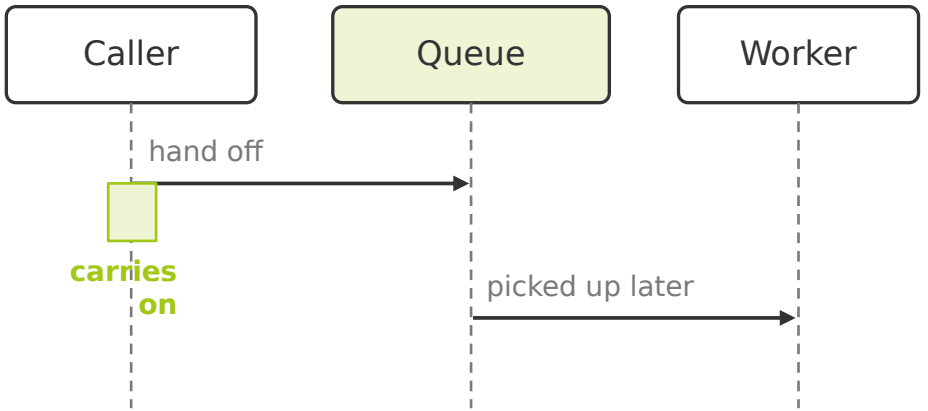
# Synchronous or asynchronous

**SYNCHRONOUS**




The caller cannot continue until the answer arrives. It needs the service to be up, reachable and fast, right now.
Simple to write. Simple to debug. Fragile under load.

**ASYNCHRONOUS**




The caller carries on immediately. The work happens when a worker is ready for it.
Absorbs load and outages. Harder to trace and to test.

**The question for every arrow on a model: does the caller have to wait?**

# Who is left waiting?

*The same question, asked of three interactions that appear in this module*

| | | | |
|---|---|---|---|
| Browser calls a Django REST endpoint | TB2 | **The user waits** | Simple to write and easy to trace. The page is only as available as the service behind it. |
| BPMN service task hands work to a job worker | TB1 | **Only the token waits** | Reads as one sequential step in the model. The engine stays free and the work can be retried elsewhere. |
| A service publishes an event | TB2 | **Nothing waits** | Sender and receiver are fully decoupled. Nobody can say when the work is finished. |

**A step can read as sequential in the model and still be asynchronous underneath.**

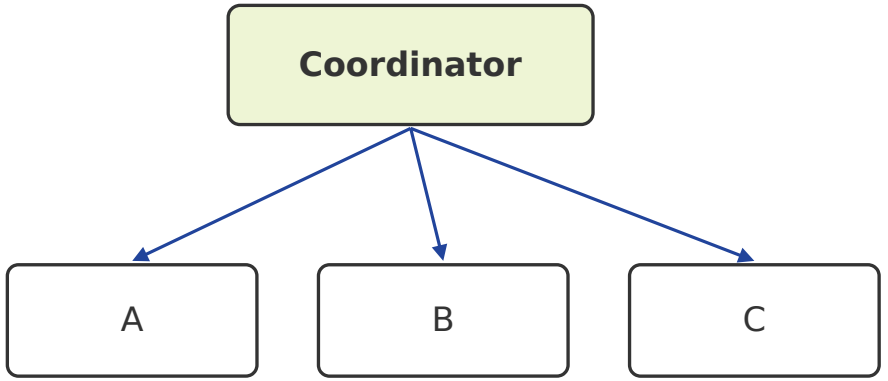
# What is really being chosen

| Time | Availability | Contract |
| --- | --- | --- |
| **Does A have to wait for B?** | **If B is down, is A down?** | **If B changes its data format, does A break?** |
| Sync couples them.<br>Async does not. | Sync usually yes.<br>Async usually no. | Both couple here.<br>Versioning is the fix. |

**Loose coupling is not free. It is paid for in complexity, every time.**

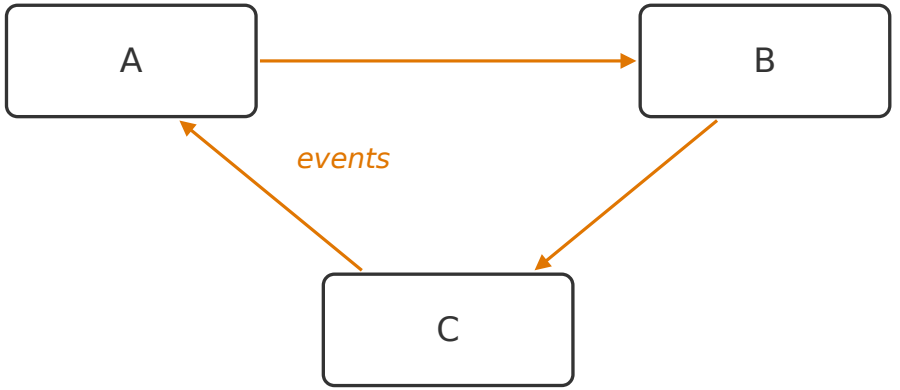
# Orchestration or choreography

**ORCHESTRATION**




One component knows the whole sequence and calls each step.
Easy to see, to change and to report on. One place to go wrong.

**CHOREOGRAPHY**



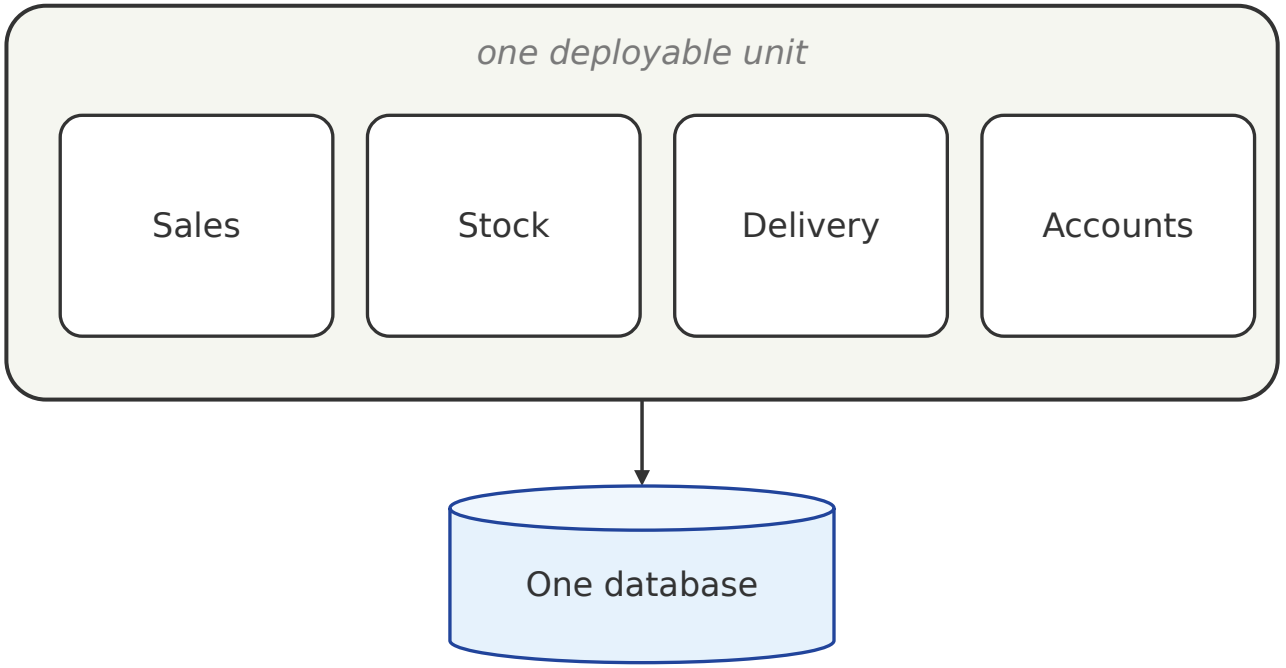

No coordinator. Each part reacts to what it hears.
Very loosely coupled. Nobody can say where an order has got to.

**BPMN makes orchestration visible. That visibility is the whole point.**

# Architectural styles

A spectrum, not a ladder

# Monolith




**Good**

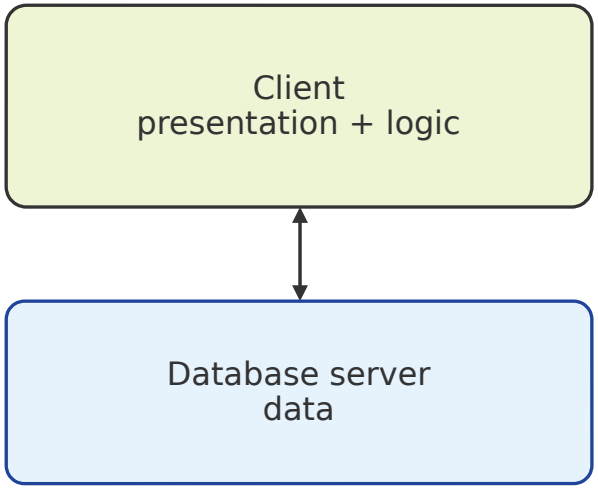
One build, one deploy, one place to look. Refactoring across modules is cheap.

**Costly**

Scales as one lump. One bad module can take the process down with it.

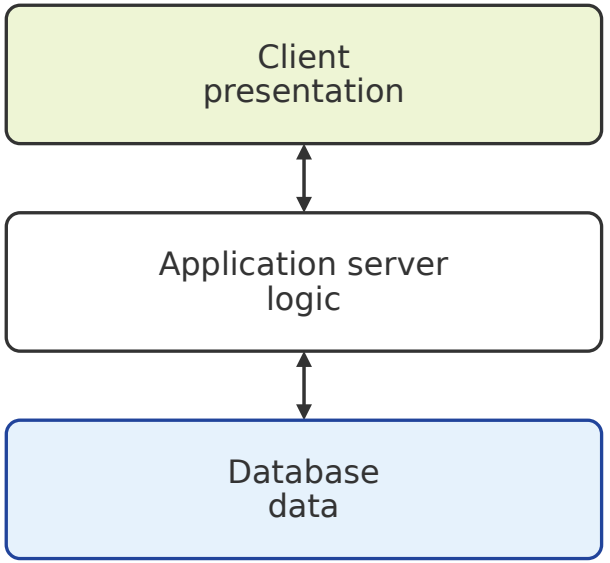
# Client-server, and tiers

**Two-tier**




Logic lives on every desktop.
Every change means touching every machine.

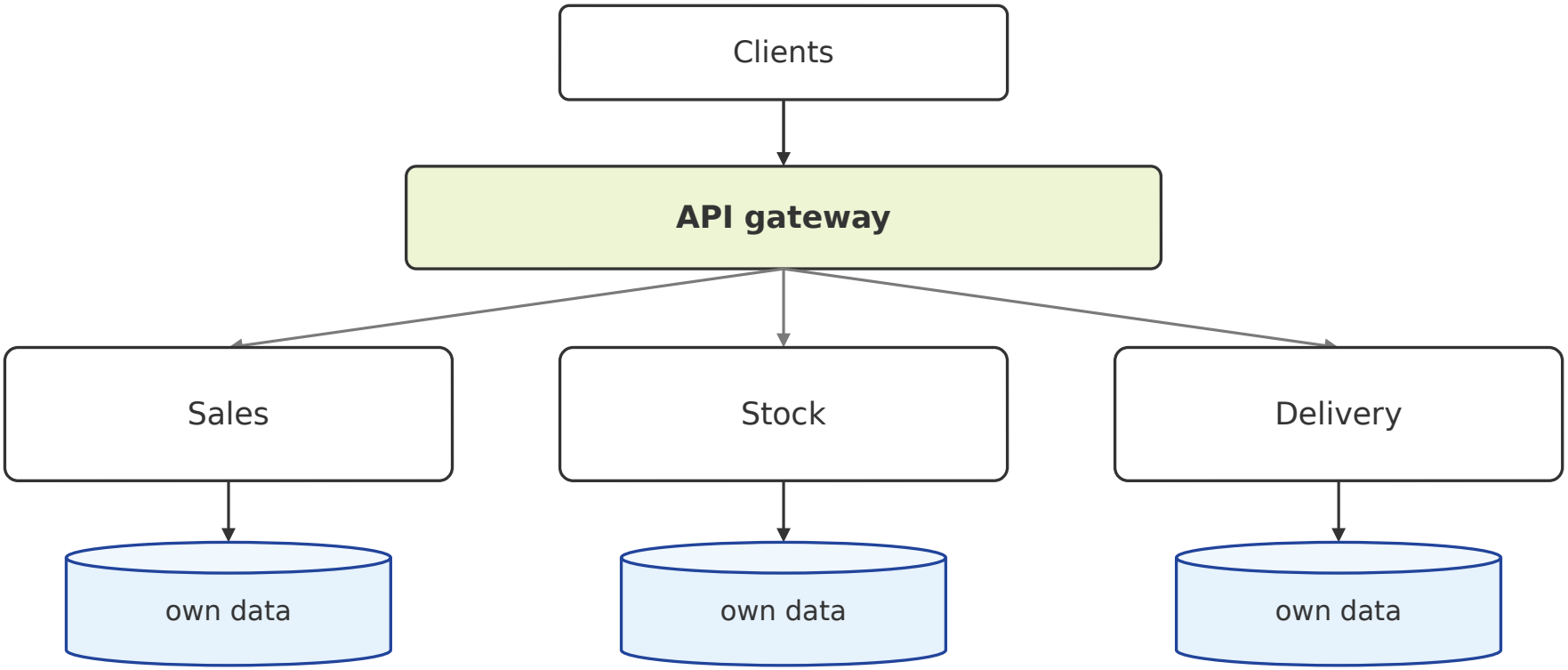
**Three-tier**




Logic sits in one place, under one team’s control.
More application servers take more load.

**A tier is a deployment boundary, not a folder in a project.**

# Services

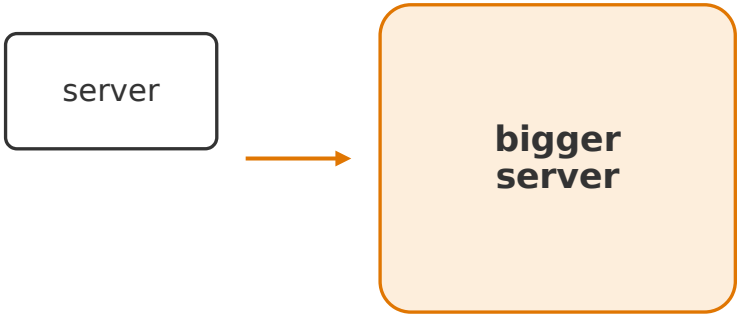



Each service is deployed, scaled, owned and can fail on its own.

**The complexity has not been removed. It has moved into the network.**

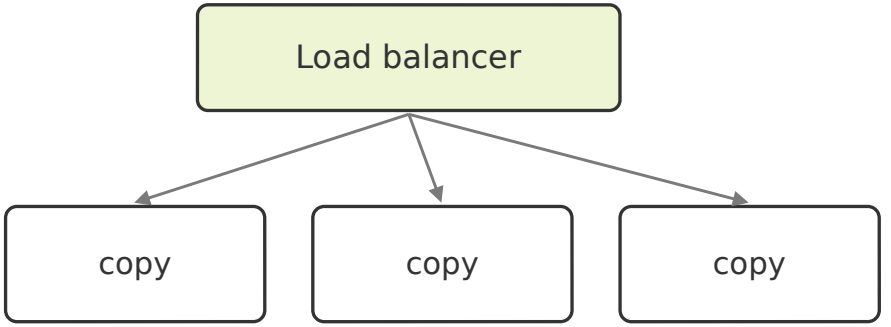
# Scaling: up or out

**SCALE UP (vertically)**




Buy a faster machine. No code changes. Works until it does not: there is a biggest machine, the price curve at the top is brutal, and it is still one thing that can fail.

**SCALE OUT (horizontally)**




Run more copies behind a load balancer. Scales much further and tolerates losing a copy, but only if the copies are interchangeable, which means holding no session state.

**Scaling out is why statelessness keeps coming up. It is the precondition.**

# There is no free lunch

**One deployable** — **Many deployables**

*monolith* ⟷ *services*

## Moving right buys

- independent deployment
- independent scaling
- independent failure
- clear team ownership

## Moving right costs

- network calls that fail
- no single transaction
- harder debugging
- more to operate

**Split when a force from Part 1 demands it. Not before.**

# Questions to ask of any design

**For any system under design: what stays together, what comes apart, and why?**

? **Which force applies?**

Scale, geography, ownership or failure isolation. Name it for each proposed split.

? **Which calls must be synchronous?**

Stock check at the till. Which ones could be asynchronous instead?

? **What happens when a dependency is down?**

Does everything stop, or does it degrade gracefully?

? **Where does the data live?**

One database, or one per service? What breaks once it is split?

*There is no single right answer. There are defensible answers and indefensible ones.*

# Data across machines

The part that does not copy neatly

# Two ways to spread data

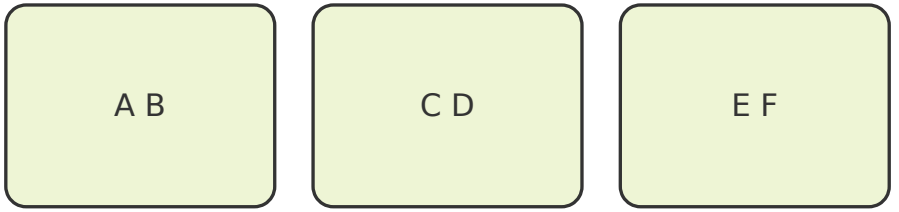
**REPLICATION**




*the same data, more than once*

Buys: any copy can serve a read, and losing a node loses no data.
Costs: a write must reach every copy, so the copies disagree until it has.

**PARTITIONING**




*different data on each node*

Buys: a dataset larger than one machine, and writes spread across nodes.
Costs: a query spanning partitions is slow, and losing a node loses that partition.

**Real systems do both: partition the data, then replicate each partition.**

# How stale is too stale?

*Once data is replicated, “the current value” stops being a single thing*

## STRONG

**Every read sees the latest write, always.**

Coordination on every write. Slower, and stops during a partition.

*Account balances, stock allocation, credit limits.*

## READ AFTER WRITE

**A writer always sees its own changes. Other readers may lag.**

A useful middle ground that avoids most of the surprise for the person acting.

*Profile edits, comments, saved drafts.*

## EVENTUAL

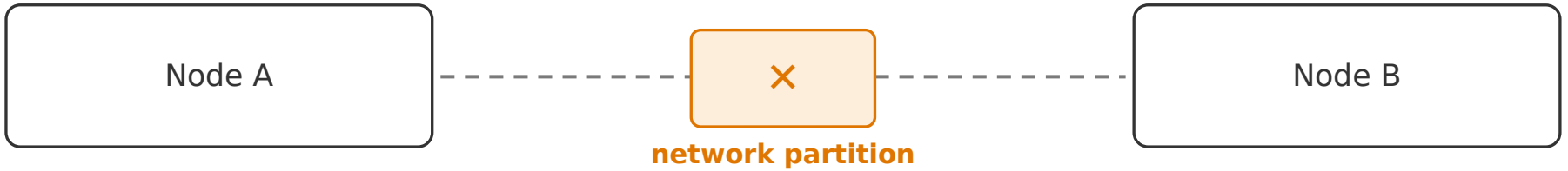
**Given no new writes, the copies converge in the end.**

Cheap and highly available. Someone will read a stale value at some point.

*View counts, recommendations, search indexes.*

**Pick the weakest model the business can actually live with.**

# CAP, stated honestly




Node B is asked for a value it can no longer verify with Node A. Two choices, no third option:

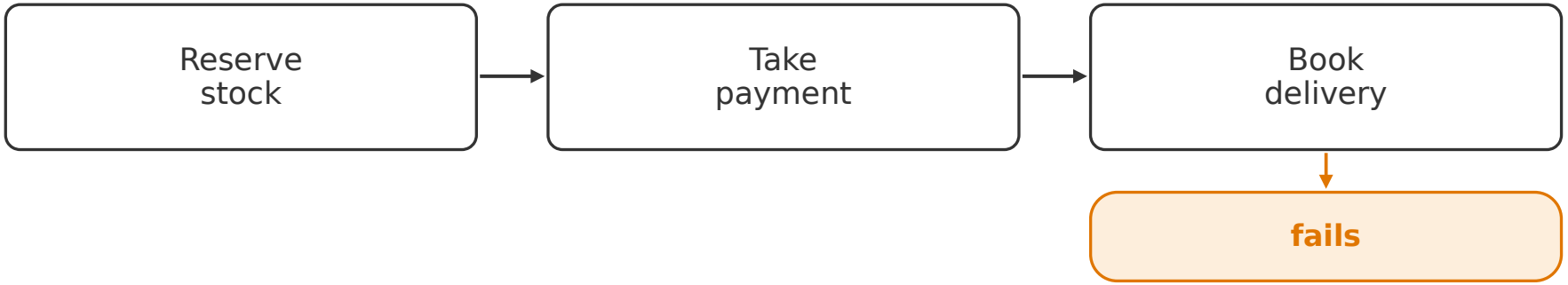
| Refuse to answer | Answer anyway |
|---|---|
| **chooses Consistency** | **chooses Availability** |
| No stale value is ever given.<br>That request cannot be served. | The request is served.<br>The value may be stale. |

**The choice only arises during a partition. The rest of the time both are available.**

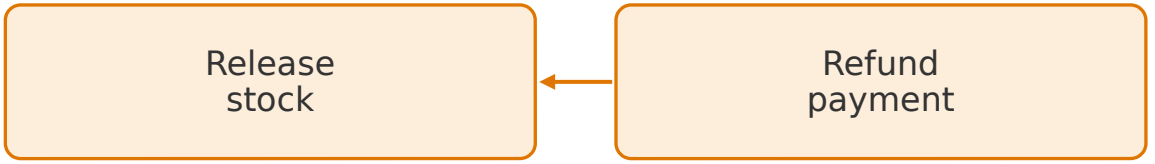
Conjectured by Eric Brewer in 2000 and proved by Seth Gilbert and Nancy Lynch in 2002.
Brewer later warned that “pick two of three” is misleading, which is why this slide avoids it.

# Transactions that span services

One COMMIT cannot cover three databases owned by three services. So the sequence becomes:




so whatever already happened is undone, in reverse:




*each step needs a compensating action*

**This is a saga, and a BPMN model is a very good place to make it visible.**

# What breaks

And what can be done about it

# The eight fallacies

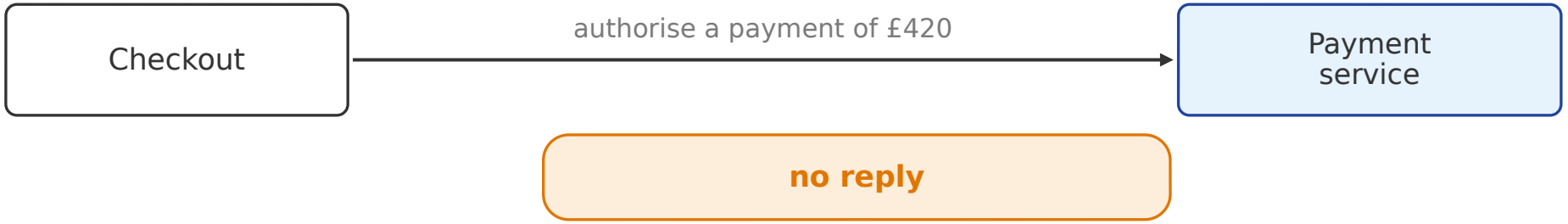
*Things every distributed system has been built on the assumption of, and none of which are true*

1. The network is reliable
2. Latency is zero
3. Bandwidth is infinite
4. The network is secure
5. Topology does not change
6. There is one administrator
7. Transport cost is zero
8. The network is homogeneous

A branch network link drops for ninety seconds every evening. Was the system designed for that, or did it quietly assume fallacy number one?

L. Peter Deutsch listed the first seven at Sun Microsystems in 1994, drawing on points raised by Bill Joy and Dave Lyon. James Gosling added the eighth in about 1997.

# Partial failure




**The caller cannot tell these apart:**

| **Never arrived** | **Arrived, still working** | **Done, reply lost** |
|---|---|---|
| the service did nothing | the answer is coming | the payment already went through |

**Timeout** decide how long the caller is willing to wait, and what happens then

**Retry** try again, with a delay, a limit, and a fallback

**Idempotency** make a repeated request safe, so a retry cannot charge twice

# Seeing inside a distributed system

*In one process there is one stack trace. Across ten services there is none.*

### Logs

**what happened**

individual events, with enough context to matter

### Metrics

**how much, how often**

counts, rates and latencies suitable for alerting

### Traces

**where the time went**

one request followed across every service it touched

The piece that makes all three usable: a correlation ID generated at the edge and passed through every call, so one customer complaint becomes one searchable thread.

**Design this in at the start. Retrofitting it during an incident is miserable.**

# Every boundary is an attack surface

**In transit**

Traffic leaves the process and can be read or altered. TLS everywhere, including between internal services.

**Authentication**

Who is calling? People and services both need identities. Tokens rather than shared passwords.

**Authorisation**

What may this caller do? Decide it at each service, not only at the gateway.

**Secrets**

Keys and credentials belong in a secret store and in configuration, never in the repository.

**Personal data**

Where it is stored and who it is shared with is a legal question, not only a technical one.

**Distributing a system multiplies the doors. Each one has to be locked separately.**

# Where this module goes next

Every tool answers one of today's problems

# Four problems, four tools

| | | |
|---|---|---|
| Nobody agrees what the process actually is | **BPMN and Camunda** | TB1, Assessment Task 1 |
| A capability has to be callable across a network | **REST and Django REST Framework** | TB2, Assessment Task 2 |
| It has to run the same everywhere, and scale | **Docker and containers** | TB2, Assessment Task 2 |
| Several people have to build it without colliding | **Scrum, sprints, version control** | All year |

**None of these are new topics. They are answers to the problems in this lecture.**

# REST, in one slide

*Six constraints. An interface that meets all the required ones is RESTful.*

**Client–server**

separate the interface from the storage

**Stateless**

every request carries everything it needs

**Cacheable**

responses say whether they may be reused

**Uniform interface**

one generic way to address and move things

**Layered system**

a client cannot tell how many hops there are

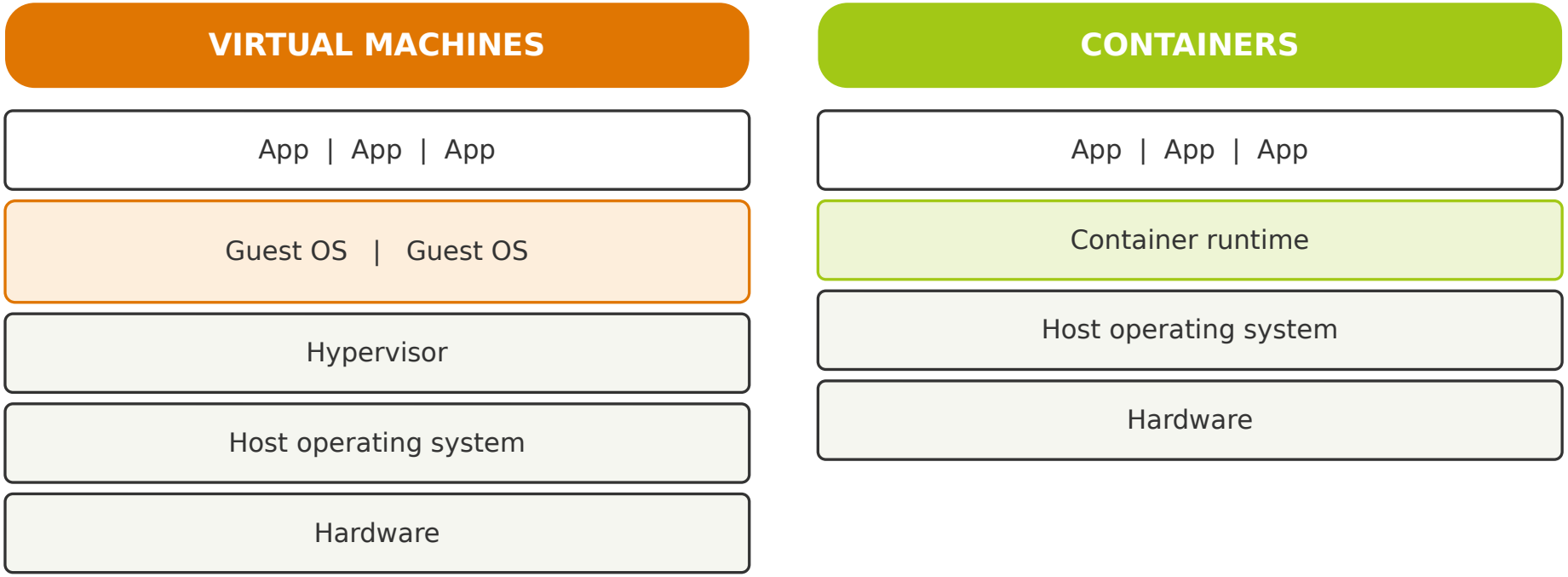
**Code on demand**

optional: the server may send code to run

Stateless is the one that matters most here: because the server keeps nothing between requests, any copy of the service can answer any request, which is exactly what makes scaling out possible.

Roy Fielding, Architectural Styles and the Design of Network-based Software Architectures, 2000.

# Containers, briefly




Each app carries a whole operating system.
Minutes to start, gigabytes on disk.

Apps share the host kernel but stay isolated.
Seconds to start, megabytes on disk.

**Cheap to start, identical everywhere, easy to replicate.**

# To summarise

### Distribution is forced, not chosen

Scale, geography, ownership or failure isolation. Name the force before the architecture.

### Every boundary is a message, and messages fail

Slow, lost, duplicated, reordered. Design for it, or it gets designed by accident.

### Synchronous or asynchronous is the recurring question

It decides how tightly two parts are coupled in time and in availability.

### Data is the hard part, not computation

Replicate it and the copies disagree. Split it and one transaction no longer covers it.

### Styles are a spectrum with an exchange rate

Independence is bought with network cost, operational cost and lost transactions.

**A design that cannot be costed is not yet understood.**

# Readings

*Current editions. The module reading list on Blackboard is the authoritative version.*

Ford, N., Richards, M., Sadalage, P. and Dehghani, Z. (2021) *Software architecture: the hard parts: modern trade-off analyses for distributed architectures*. O'Reilly Media.
*Trade-off analysis for distributed architectures, including sagas.*

Kleppmann, M. and Riccomini, C. (2026) *Designing data-intensive applications*. 2nd edn. O'Reilly Media.
*Replication, partitioning and consistency in depth. The text for Part 4.*

Newman, S. (2021) *Building microservices: designing fine-grained systems*. 2nd edn. O'Reilly Media.
*Service boundaries, and when not to draw them.*

Richards, M. and Ford, N. (2025) *Fundamentals of software architecture: a modern engineering approach*. 2nd edn. O'Reilly Media.
*Architectural styles and how to decide between them.*

van Steen, M. and Tanenbaum, A.S. (2023) *Distributed systems*. 4th edn. Available at: https://www.distributed-systems.net/index.php/books/ds4/ (Accessed: 21 September 2026).
*The academic text. A digital copy is offered free by the authors.*

# References


Brewer, E. (2012) ‘CAP twelve years later: how the “rules” have changed’, *Computer*, 45(2), pp. 23–29.

Camunda (2026) *Service tasks*. Available at: https://docs.camunda.io/docs/components/modeler/bpmn/service-tasks/ (Accessed: 21 September 2026).

Fielding, R.T. (2000) *Architectural styles and the design of network-based software architectures*. PhD thesis. University of California, Irvine. Available at: https://ics.uci.edu/~fielding/pubs/dissertation/top.htm (Accessed: 21 September 2026).

Gilbert, S. and Lynch, N. (2002) ‘Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services’, *ACM SIGACT News*, 33(2), pp. 51–59.

Majors, C., Fong-Jones, L. and Miranda, G. (2026) *Observability engineering: achieving production excellence*. 2nd edn. O’Reilly Media.

Object Management Group (2013) *Business process model and notation (BPMN) version 2.0.2*. Available at: https://www.omg.org/spec/BPMN/2.0.2/ (Accessed: 21 September 2026).

OpenTelemetry (2026) *OpenTelemetry documentation*. Available at: https://opentelemetry.io/docs/ (Accessed: 21 September 2026).

Rotem-Gal-Oz, A. (2006) *Fallacies of distributed computing explained*. Available at: https://arnon.me/wp-content/uploads/Files/fallacies.pdf (Accessed: 21 September 2026).

Ruecker, B. (2021) *Practical process automation: orchestration and integration in microservices and cloud native architectures*. O’Reilly Media.

van Steen, M. and Tanenbaum, A.S. (2023) *Distributed systems*. 4th edn. Available at: https://www.distributed-systems.net/index.php/books/ds4/ (Accessed: 21 September 2026).

Yang, M., Wang, X., Cai, B., Wang, Y. and Guo, Y. (2025) ‘Full stack optimization of microservice architecture: systematic review and research opportunity’, *Cluster Computing*, 28, article 1005. https://doi.org/10.1007/s10586-025-05690-6